INFRASTRUCTURE AS CODE — TP

Docker & Terraform

Architecture Microservices — Déploiement local

Docker | Terraform | Provider kreuzwerker/docker | State | Modules | Variables | Locals | for_each

MODALITÉ
Binômes — rendu Git

DURÉE
6 h (3 séances)

PRÉREQUIS
Docker Desktop installé, CLI Terraform ≥ 1.6

RENDU ATTENDU
Dépôt Git + démo fonctionnelle

Aucun compte cloud requis. Toute l'infrastructure tourne localement via Docker.
Commande de nettoyage à la fin de chaque séance : `terraform destroy`

Contexte & Architecture cible

Le projet **ShopMicro** est une application e-commerce décomposée en quatre services indépendants. Votre objectif est de déployer et gérer cette architecture localement en utilisant **Terraform avec le provider Docker**, sans toucher à Docker Compose une fois la Partie 1 terminée.

```
┌──────────────────────────────────────────┐
│        Machine hôte (localhost)          │
│                                          │
│  :8080        :3001    :3002    :3003    │
│    │           │        │        │       │
└────┼───────────┼────────┼────────┼───────┘
     │           │        │        │
┌────▼─┐   ┌────▼─┐  ┌───▼────┐  ┌─▼──────┐
│ frontend │  │  user  │  │product │  │  order  │
│  :80     │  │ :3001  │  │ :3002  │  │ :3003   │
└──────────┘  └───┬────┘  └───┬────┘  └────┬────┘
                  │           │            │
                  └─────┬─────┘            │
                        │   ◄──────────────┘
                ┌───────▼────────┐
                │   PostgreSQL   │
                │    :5432       │
                └────────────────┘

     (tous sur le même réseau Docker bridge)
```

Les fichiers sources Node.js, les Dockerfiles et les fichiers Terraform de base sont fournis. **Lisez-les entièrement avant de commencer.**

Partie 1 — Docker : maîtrise de l'existant

01 Lecture du code source

Avant toute chose, lisez les trois fichiers `src/index.js`.

1. Dressez le tableau exhaustif de toutes les routes HTTP exposées par chaque service (méthode, chemin, comportement attendu).
2. Le `order-service` appelle les deux autres services. Identifiez exactement quels appels sont faits, dans quel ordre, et ce qui se passe si l'un d'eux échoue avec un statut 404.
3. Chaque service a deux modes de fonctionnement selon la présence ou non de `DATABASE_URL`. Expliquez la différence de comportement et pourquoi ce pattern est utile pendant le développement.

RÉPONSE

02 Docker Compose — démarrage et vérification

1. Démarrez l'application avec `docker compose up --build -d`. Vérifiez que les cinq conteneurs sont actifs.
2. Sans utiliser le navigateur, créez un utilisateur, un produit, puis une commande via `curl`. Écrivez les commandes utilisées.
3. Tentez de créer une commande avec un `user_id` inexistant. Quel code HTTP obtenez-vous et pourquoi ?
4. Stoppez uniquement le conteneur `user-service`. Que se passe-t-il si vous tentez de créer une commande ? Relancez le service et expliquez pourquoi `restart: unless-stopped` ne suffit pas dans ce cas.

RÉPONSE

03 Analyse des Dockerfiles et optimisation

1. Examinez les trois Dockerfiles. Quelle est la différence entre `npm install` et `npm ci` ? Pourquoi l'un est-il préféré dans un contexte d'image de production ?
2. Lancez `docker images | grep shopmicrfo` (après build via Docker Compose). Notez les tailles. Proposez et implémentez une modification des Dockerfiles qui réduit la taille des images. Mesurez l'impact.
3. Ajoutez un fichier `.dockerignore` dans chaque dossier de service. Quels fichiers doit-il exclure et pourquoi ?
4. La directive `HEALTHCHECK` est présente dans les Dockerfiles. Quelle est la différence entre l'état `healthy`, `unhealthy` et `starting` d'un conteneur ? Comment le vérifier ?

RÉPONSE

04 Réseaux Docker

1. Identifiez le réseau Docker créé par Docker Compose pour ce projet. Quel driver est utilisé ? Quel sous-réseau est assigné ?
2. Exécutez un shell dans le conteneur `order-service`. Pouvez-vous pinguer le conteneur `postgres` par son nom ? Par son IP ? Expliquez le mécanisme DNS interne de Docker.
3. Dans `docker-compose.yml`, les services n'ont pas de configuration `networks` explicite. Comment Docker Compose détermine-t-il le réseau à créer ?
4. Créez manuellement un réseau Docker `monitoring_net`. Ajoutez dans le `docker-compose.yml` un service **Adminer** (interface web PostgreSQL) sur le port `8081`, connecté à la fois au réseau de l'application et à `monitoring_net`.

Partie 2 — Terraform : prise en main du provider Docker

05 Initialisation et exploration

Stoppez Docker Compose (`docker compose down -v`). Désormais, vous n'utiliserez plus Docker Compose — Terraform prend le relais.

1. Dans le dossier `terraform/` , listez et décrivez brièvement le rôle de chaque fichier `.tf` .
2. Lancez `terraform init` . Que télécharge cette commande ? Où les fichiers sont-ils placés ?
3. Lancez `terraform validate` . Corrigez les erreurs éventuelles.
4. Lancez `terraform plan` . Relevez : combien de ressources vont être créées ? Quels sont les types de ressources distincts ? Y a-t-il des ressources qui dépendent d'autres ? Comment le distinguer dans la sortie du plan ?

RÉPONSE

06 Premier apply et vérification

1. Lancez `terraform apply` . Pendant l'exécution, observez dans un autre terminal avec `docker ps -a` l'ordre d'apparition des conteneurs. Correspond-il à ce que vous attendiez des dépendances ?
2. Une fois l'apply terminé, récupérez les outputs avec `terraform output` . Vérifiez que l'application est fonctionnelle via les URLs affichées.
3. Ouvrez le fichier `terraform.tfstate` . Identifiez la structure JSON : que représente un bloc `resource` dans le state ? Quelle information cruciale y trouve-t-on par rapport au code source ?
4. Lancez `terraform plan` une seconde fois sans rien changer. Que constatez-vous ? Expliquez pourquoi.

RÉPONSE

07 Variables, locals et expressions

1. Dans `variables.tf`, la variable `db_password` est marquée `sensitive = true`. Modifiez sa valeur via la ligne de commande (`-var`) et observez la différence dans le plan. Que cache Terraform exactement ?
2. Ajoutez une variable `restart_policy` de type `string` avec une contrainte de validation n'acceptant que les valeurs `"always"`, `"unless-stopped"` et `"on-failure"`. Utilisez-la dans tous les conteneurs de `services.tf`.
3. Dans `locals.tf`, la variable `db_url` est construite dynamiquement. Expliquez pourquoi une référence à `docker_container.postgres.name` crée une dépendance implicite. Vérifiez-le dans le graphe de dépendances (`terraform graph`).
4. Ajoutez un local `all_service_ports` qui construit une map associant le nom de chaque service à son port hôte. Utilisez-le dans un output `services_ports`.

RÉPONSE

08 Dépendances implicites et explicites

1. Dans `services.tf`, l' `order-service` a un `depends_on` explicite sur postgres. Est-il réellement nécessaire ? Justifiez en traçant le chemin de dépendances implicites existantes.
2. Supprimez le bloc `depends_on` de `order-service`. Lancez `terraform plan` puis `terraform graph`. Le comportement change-t-il ?
3. Créez un scénario où un `depends_on` explicite est **indispensable** — c'est-à-dire où aucune référence directe ne crée la dépendance nécessaire. Décrivez ce scénario.

RÉPONSE

Partie 3 — State Terraform

09 Inspection et manipulation du state

1. Listez toutes les ressources du state avec `terraform state list`. Pour chaque type de ressource (`docker_image`, `docker_container`, etc.), expliquez ce que Terraform trace dans le state.
2. Affichez le state complet du conteneur postgres : `terraform state show docker_container.postgres`. Relevez trois attributs présents dans le state qui ne sont définis nulle part dans le code `.tf`. D'où viennent-ils ?
3. **Scénario de dérive :** Stoppez et supprimez manuellement le conteneur `user_service` avec `docker rm -f ...`. Lancez ensuite `terraform plan`. Que constate Terraform ? Quelle action va-t-il prendre et pourquoi ?
4. Lancez `terraform apply` pour corriger la dérive. Vérifiez que le conteneur est recréé.

RÉPONSE

10 terraform import

1. Créez manuellement un conteneur Docker nommé `shopmicrfo_adminer` à partir de l'image `adminer:latest`, exposé sur le port `8081` et connecté au réseau `shopmicrfo_network`.
2. Écrivez dans un nouveau fichier `adminer.tf` la ressource Terraform correspondant à ce conteneur.
3. Importez le conteneur dans le state Terraform avec `terraform import`. Consultez la documentation du provider pour trouver l'identifiant à utiliser.
4. Lancez `terraform plan`. Y a-t-il des différences entre l'état importé et votre code ? Corrigez-les jusqu'à obtenir un plan vide.

RÉPONSE

11 Cycle de vie et terraform taint

1. Dans le module `microservice`, un bloc `lifecycle { ignore_changes = [image] }` est présent. Simulez le cas où l'image du `user-service` est mise à jour (modifiez le code, rebuildez). Que se passe-t-il lors du `terraform plan` avec ce bloc ? Sans ce bloc ?
2. Utilisez `terraform taint docker_container.user_service` (ou son équivalent moderne). Que fait cette commande ? Vérifiez avec `terraform plan`.
3. Ajoutez un bloc `lifecycle { create_before_destroy = true }` sur le conteneur postgres. Quel impact cela a-t-il sur la procédure de remplacement du conteneur ? Pourquoi est-ce utile pour un service avec état (stateful) ?

RÉPONSE

Partie 4 — count, for_each et expressions avancées

12 count vs for_each

1. Actuellement, chaque conteneur de service est une ressource distincte dans `services.tf`. Réécrivez les trois conteneurs de microservices (user, product, order) en utilisant **une seule ressource** `docker_container` avec `for_each` itérant sur le local `services` défini dans `locals.tf`.
2. Quelle est la différence dans le state entre une ressource créée avec `count = 3` et une ressource créée avec `for_each` sur un map de 3 éléments ? Illustrez avec les adresses de ressources Terraform.
3. Vous retirez un service du map `for_each`. Que se passe-t-il ? Même question avec `count` si vous retirez un élément au milieu de la liste.
4. La variable d'environnement `DATABASE_URL` est commune à tous les services, mais `USER_SERVICE_URL` et `PRODUCT_SERVICE_URL` sont spécifiques à l'order-service. Comment gérez-vous cette différence dans votre ressource `for_each` ?

RÉPONSE

13 Expressions : fonctions et conditions

1. Ajoutez une variable booléenne `expose_postgres` (défaut : `true`). Utilisez une expression conditionnelle dans le bloc `ports` du conteneur postgres pour n'exposer le port `5432` sur l'hôte que si cette variable est `true`. Vérifiez les deux cas avec `terraform plan`.
2. Dans `images.tf`, le hash de déclenchement de rebuild est construit avec `sha256(join(...))`. Expliquez le rôle de ce mécanisme. Que se passe-t-il si vous modifiez `src/index.js` du user-service et relancez `terraform apply` ?
3. Utilisez la fonction `toset()` pour créer un volume Docker par service (user, product, order) en une seule ressource `docker_volume` avec `for_each`. Ces volumes resteront vides pour l'instant.

RÉPONSE

Partie 5 — Modules

14 Analyse du module microservice existant

1. Examinez les trois fichiers du module `modules/microservice/`. Listez tous ses inputs et outputs. Que ne peut-il pas gérer que la définition directe dans `services.tf` gère ?
2. Réécrivez **un seul service** (au choix) dans `services.tf` pour qu'il utilise ce module au lieu d'une ressource directe. Vérifiez avec `terraform plan` que le résultat est un remplacement (ou idéalement aucune modification si les attributs sont identiques).
3. Le module contient un `lifecycle { ignore_changes = [image] }`. Quand cette règle est-elle problématique ? Proposez une solution pour permettre de forcer le remplacement de l'image depuis l'appelant du module sans modifier le module lui-même.

RÉPONSE

15 Créer un module complet

Créez un module `modules/service-stack/` qui regroupe : le build de l'image Docker, la création du conteneur, et un volume optionnel pour ce service.

1. Définissez les inputs : `name`, `build_context`, `dockerfile_path`, `internal_port`, `external_port`, `network_name`, `env_vars`, `project_name`, `enable_volume` (booléen, défaut `false`).
2. Le volume ne doit être créé que si `enable_volume = true`. Utilisez `count` pour ce comportement conditionnel.
3. Exposez en output : `container_name`, `container_id`, `image_id`, `volume_name` (null si pas de volume).
4. Remplacez les ressources du `user-service` dans le code principal par un appel à ce module. Vérifiez que le plan ne crée aucune ressource supplémentaire.

RÉPONSE

Partie 6 — Mise à jour, destroy et cas avancés

16 Mise à jour d'un service sans interruption

1. Modifiez le `user-service` : ajoutez un champ `role` (défaut : `"user"`) dans la route `POST /users` et renvoyez-le dans les réponses `GET`. Vérifiez localement que la modification fonctionne.
2. Mettez à jour le tag de l'image en changeant `services_image_tag` à `"v2"` dans `terraform.tfvars`. Lancez `terraform plan` : quelles ressources sont affectées ? Pourquoi plus que juste le conteneur ?
3. Avec le `lifecycle { ignore_changes = [image] }` actif dans le module, expliquez pourquoi le conteneur **ne sera pas recréé** même si l'image change. Comment forcer la recréation ?
4. Après l'apply, vérifiez que la nouvelle version du service répond correctement. Utilisez `docker inspect` pour confirmer quelle image est utilisée par le conteneur.

RÉPONSE

17 Gestion de la destruction et des données persistantes

1. Insérez quelques utilisateurs et produits dans l'application. Lancez `terraform destroy`. Que devient le volume `postgres_data` ? Les données sont-elles perdues ?
2. Relancez `terraform apply`. Les données sont-elles restaurées ? Expliquez en fonction du comportement du provider Docker sur les volumes.
3. Ajoutez un bloc `lifecycle { prevent_destroy = true }` sur la ressource `docker_volume.postgres_data`. Testez `terraform destroy`. Que se passe-t-il ? Comment contourner cette protection quand c'est vraiment nécessaire ?
4. Supprimez **uniquement** les conteneurs sans toucher au volume ni aux images. Écrivez la ou les commandes `terraform` permettant de cibler des ressources spécifiques.

RÉPONSE

18 Provisioners et scripts d'initialisation

1. Consultez la documentation Terraform sur les **provisioners**. Quelle est la philosophie officielle de Terraform à leur égard ?
2. Ajoutez un provisioner `local-exec` sur le conteneur postgres qui attend que la base soit prête avant de continuer (utilisez `docker exec ... pg_isready`). Ce provisioner doit s'exécuter à la **création** uniquement.
3. Testez le comportement : détruisez le conteneur postgres et relancez `terraform apply`. Le provisioner est-il exécuté lors de la recréation ?
4. Pourquoi les provisioners sont-ils considérés comme un "last resort" en Terraform ? Proposez une alternative pour l'initialisation de schéma de base de données.

RÉPONSE

Aide-mémoire — Commandes & Concepts

Docker — commandes essentielles

Commande	Ce qu'elle fait
`docker compose up --build -d`	Build les images depuis les Dockerfiles puis démarre tous les services définis en arrière-plan (`-d` = detached).
`docker compose down`	Stoppe et supprime les conteneurs. Les volumes nommés sont conservés.
`docker compose down -v`	Idem + supprime aussi les volumes. Les données persistantes sont perdues.
`docker compose logs -f <service>`	Affiche et suit en temps réel les logs d'un service spécifique.
`docker compose ps`	Liste l'état de chaque service (Running, Exited, healthy…).
`docker ps -a`	Liste tous les conteneurs Docker actifs et stoppés sur la machine.
`docker exec -it <nom> sh`	Ouvre un shell interactif à l'intérieur d'un conteneur actif.
`docker inspect <nom>`	Affiche la configuration complète d'un conteneur (réseau, volumes, variables d'env…).
`docker network ls`	Liste tous les réseaux Docker existants.
`docker network inspect <nom>`	Affiche les détails d'un réseau : sous-réseau, conteneurs connectés, gateway.
`docker images`	Liste toutes les images locales avec leur taille.
`docker rmi <image>`	Supprime une image locale.

Terraform — cycle de vie

Commande	Ce qu'elle fait
`terraform init`	Initialise le répertoire de travail : télécharge le provider déclaré dans `required_providers` dans `.terraform/` et crée le fichier de verrou `.terraform.lock.hcl`.
`terraform validate`	Vérifie la syntaxe et la cohérence des fichiers `.tf` sans contacter Docker ni lire le state.
`terraform fmt`	Reformate automatiquement le code selon le style officiel Terraform. À lancer avant chaque commit.

Commande	Ce qu'elle fait
`terraform fmt --check`	Vérifie le formatage sans modifier les fichiers. Retourne un code d'erreur si le code n'est pas formaté.
`terraform plan`	Compare le state actuel avec le code et affiche les actions prévues (création, modification, destruction) sans rien exécuter. **À toujours faire avant apply.**
`terraform plan -out=plan.tfplan`	Sauvegarde le plan dans un fichier pour un apply garanti identique.
`terraform apply`	Exécute les actions planifiées. Demande une confirmation interactive sauf avec `-auto-approve`.
`terraform apply plan.tfplan`	Applique exactement le plan sauvegardé, sans confirmation.
`terraform destroy`	Détruit toutes les ressources gérées par le state courant. Demande confirmation.
`terraform destroy -target=<res>`	Détruit uniquement la ressource ciblée (ex : `docker_container.postgres`).

Terraform — inspection du state

Commande	Ce qu'elle fait
`terraform state list`	Liste toutes les ressources actuellement trackées dans le state (format `type.nom`).
`terraform state show <res>`	Affiche tous les attributs connus d'une ressource dans le state : ID, IP, configuration réelle.
`terraform output`	Affiche les valeurs des outputs déclarés dans `outputs.tf` après un apply.
`terraform output -json`	Même chose au format JSON, utile pour scripting.
`terraform graph`	Génère le graphe de dépendances en format DOT (visualisable sur `dreampuf.github.io/GraphvizOnline`).
`terraform import <res> <id>`	Importe une ressource existante (créée hors Terraform) dans le state sans la recréer.
`terraform taint <res>`	Marque une ressource comme à recréer lors du prochain apply, même si aucune modification n'est détectée.
`terraform refresh`	Met à jour le state en relisant l'état réel des ressources. Utilisé pour détecter une dérive.

Concepts clés

STATE Le fichier terraform.tfstate

Terraform stocke l'état réel de l'infrastructure dans un fichier JSON appelé **state**. Il contient les IDs, IPs et attributs de chaque ressource après création. C'est la source de vérité de Terraform : lors d'un `plan`, Terraform compare le code avec le state pour savoir ce qui a changé.

Ne jamais modifier le state à la main. Utiliser `terraform state mv`, `terraform state rm` ou `terraform import` selon le besoin.

En équipe, le state doit être partagé via un **backend distant** (HTTP, S3, Terraform Cloud…) pour que tous les membres voient le même état. Un mécanisme de verrou empêche deux personnes d'appliquer simultanément.

DÉRIVE Drift — quand la réalité s'écarte du code

Une **dérive** se produit quand une ressource est modifiée ou supprimée manuellement hors de Terraform (via l'interface Docker, la console cloud…). Au prochain `terraform plan`, Terraform détecte la différence entre le state et la réalité et propose de la corriger.

Exemple : si on supprime un conteneur avec `docker rm`, Terraform le recréera au prochain `apply` pour revenir à l'état désiré.

DÉPENDANCES Implicites vs explicites

Implicite : Quand une ressource A référence un attribut d'une ressource B (`docker_network.app.name`), Terraform infère automatiquement que B doit être créée avant A. C'est la forme la plus courante et recommandée.

Explicite : Le bloc `depends_on = [resource.B]` force un ordre quand il n'y a aucune référence directe. À utiliser seulement quand nécessaire, car il rend le graphe plus difficile à lire.

COUNT / FOR_EACH Créer plusieurs ressources similaires

`count = N` — crée N instances identiques d'une ressource, accessibles via `resource.name[0]`, `[1]`… Fragile : si on retire un élément au milieu de la liste, Terraform recréera toutes les ressources suivantes.

`for_each = map` — crée une instance par entrée d'un map ou d'un set. Chaque instance est adressée par sa clé (`resource.name["clé"]`). Stable : retirer une entrée ne touche que la ressource correspondante.

Règle : Toujours préférer `for_each` sur des maps nommés dès que possible.

LIFECYCLE Contrôler le comportement lors des mises à jour

- `ignore_changes = [attr]` — Terraform ignore les différences sur cet attribut. Utile quand Docker ou un autre outil modifie un attribut hors du contrôle de Terraform (ex : gateway assignée automatiquement).
- `prevent_destroy = true` — Terraform refuse de détruire cette ressource, même lors d'un `terraform destroy`. Protection contre les suppressions accidentelles (volumes de données, bases de production).
- `create_before_destroy = true` — Terraform crée le remplaçant avant de détruire l'ancien. Utile pour les services avec état où une interruption serait problématique.

MODULES Encapsuler et réutiliser

Un module est un dossier de fichiers `.tf` exposant des **variables** (inputs) et des **outputs**. Il s'appelle depuis un autre fichier avec `module "nom" { source = "./chemin" }`.

Un module bien conçu est autonome, documenté (descriptions des variables), et ne contient pas de valeurs en dur. Il doit pouvoir être instancié plusieurs fois avec des paramètres différents.

Le Terraform Registry (`registry.terraform.io`) propose des modules communautaires maintenus pour les fournisseurs courants (AWS, Docker, Kubernetes…).

VARIABLES Types, valeurs par défaut, validation

Les variables Terraform sont déclarées dans `variables.tf`. Leur valeur peut être passée par :

- Fichier `terraform.tfvars` (chargé automatiquement)
- Flag `-var="nom=valeur"` sur la ligne de commande
- Variable d'environnement `TF_VAR_nom`
- Valeur par défaut dans la déclaration (`default = ...`)

Le bloc `validation` permet de rejeter des valeurs invalides avec un message d'erreur explicite, avant même que Terraform contacte l'infrastructure.

Les variables `sensitive = true` ne sont jamais affichées dans les logs de plan/apply.

LOCALS Valeurs calculées réutilisables

Les **locals** (déclarés dans un bloc `locals { }`) permettent de nommer des expressions complexes pour les réutiliser sans répétition. Contrairement aux variables, ils ne sont pas exposés à l'extérieur du module et ne peuvent pas être surchargés depuis la ligne de commande.

Exemple typique : construire une URL de connexion à partir de plusieurs variables (`host` , `port` , `user`), puis référencer ce local dans plusieurs ressources.

Partie 7 — Exercice intégrateur

Cet exercice mobilise l'ensemble des compétences du TP. Aucune indication sur la démarche n'est fournie : à vous de concevoir la solution et d'organiser votre code.

19 Nouveau microservice : notification-service

Vous devez ajouter un cinquième microservice au projet : le **notification-service**. Ce service écoute sur le port `3004` et expose les routes suivantes :

- `GET /health` — renvoie `{"status":"ok","service":"notification-service"}`
- `POST /notify` — reçoit un body `{"user_id": N, "message": "..." }`, vérifie que l'utilisateur existe via le `user-service`, stocke la notification en base de données (table `notifications` : `id`, `user_id`, `message`, `sent_at`), et retourne la notification créée.
- `GET /notifications` — retourne la liste de toutes les notifications.
- `GET /notifications/:user_id` — retourne les notifications d'un utilisateur donné.

1. **Application :** Créez le dossier `services/notification-service/` avec son `src/index.js`, son `package.json`, son `Dockerfile` et son `package-lock.json`. Le service doit fonctionner en mode mémoire (sans `DATABASE_URL`) et en mode PostgreSQL.
2. **Docker Compose :** Ajoutez le service dans `docker-compose.yml` avec les dépendances correctes. Vérifiez avec `docker compose up --build` que les cinq services démarrent et que les health checks passent.
3. **Terraform :** Ajoutez dans le dossier `terraform/` toutes les ressources nécessaires pour ce service en utilisant le module `modules/microservice`. Pensez à : l'image Docker, le conteneur, la variable de port, l'output. Vérifiez que `terraform plan` affiche exactement 2 nouvelles ressources (image + conteneur) et que `terraform apply` démarre le service correctement.
4. **Test de bout en bout :** Via `curl`, créez un utilisateur, envoyez deux notifications à cet utilisateur, puis récupérez uniquement ses notifications. Montrez les commandes et réponses.
5. **Dérive volontaire :** Après l'apply, renommez manuellement le conteneur `notification-service` en modifiant son label avec `docker rename`. Lancez `terraform plan`. Que se passe-t-il ? Pourquoi Terraform ne détecte-t-il pas le changement de nom ? Quelle commande permet de forcer la resynchronisation ?
6. **Refactoring :** Le code de création de table PostgreSQL est dupliqué dans tous les services (appel à `pool.query(CREATE TABLE IF NOT EXISTS ...)` dans `initDb()`). Proposez une architecture alternative pour centraliser la gestion du schéma de base de données. Vous n'avez pas à l'implémenter, mais décrivez la solution et ses avantages.

RÉPONSE

20 Analyse de sécurité du code Terraform

Relisez l'ensemble du code Terraform produit pendant le TP et répondez aux questions suivantes.

1. Le mot de passe PostgreSQL est déclaré comme variable `sensitive = true` et stocké dans `terraform.tfvars`. Ouvrez le fichier `terraform.tfstate` après un apply. Le mot de passe y est-il présent ? En clair ou chiffré ? Quelle conclusion en tirez-vous sur la protection du state ?
2. Le conteneur PostgreSQL expose le port `5432` sur l'hôte (`external = 5432`). Depuis un autre processus sur la machine hôte, est-il possible de se connecter à cette base sans passer par les conteneurs applicatifs ? Quel risque cela représente-t-il ? Comment le corriger dans Terraform ?
3. Dans `images.tf`, le trigger de rebuild est calculé avec `sha256(join(...))` sur trois fichiers. Quel fichier important lié aux dépendances Node.js n'est pas inclus dans ce calcul ? Quelles conséquences cela peut-il avoir ? Corrigez le trigger.
4. La variable `db_password` a la valeur par défaut `"shoppassword"` dans `terraform.tfvars`. Proposez un mécanisme Terraform permettant d'obliger l'opérateur à fournir explicitement un mot de passe en production, sans valeur par défaut possible.

RÉPONSE

Synthèse & Évaluation

Partie	Thème	Exercices	Points
1	Docker — maîtrise de l'existant	01 → 04	4
2	Terraform — prise en main provider Docker	05 → 08	5
3	State Terraform	09 → 11	5
4	count, for_each, expressions	12 → 13	4
5	Modules	14 → 15	5
6	Mise à jour, destroy, provisioners	16 → 18	4
7	Exercice intégrateur — nouveau service + sécurité	19 → 20	6
Qualité du code (lisibilité, nommage, fmt)			**3**
Total			**36**

Critères de rendu

1. Dépôt Git avec un historique de commits significatif (un commit par exercice ou groupe logique).
2. Le code passe `terraform validate` et `terraform fmt --check` sans erreur.
3. `terraform plan` produit un plan vide (no changes) sur une infra déjà déployée.
4. L'application est fonctionnelle : création d'utilisateur, produit et commande via le frontend ou curl.
5. Aucun mot de passe en clair dans l'historique Git.

> **Ressources autorisées :** Documentation officielle Terraform (`registry.terraform.io`), documentation du provider `kreuzwerker/docker` , documentation Docker. Le code doit être compris et rédigé par le binôme.